VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF COMPUTER NETWORKS AND COMMUNICATIONS

TECHNICAL INTERNSHIP REPORT

Automated Infrastructure and GitOps Delivery for Microservices on Azure

Xelo Investment Consultancy Vietnam

UNIVERSITY ADVISOR: Le Anh Tuan

COMPANY MENTOR: Tran Van Viet

STUDENT NAME: Le Trung Kien

STUDENT ID: 23520797

HO CHI MINH CITY, 2026

Abstract

This report summarizes the technical work conducted during my internship at Xelo Investment Consultancy Vietnam, focusing on the deployment and infrastructure automation for MyTask-Tracker, a microservices-based application. The primary objective was to transition the system from manual deployments to a fully automated, GitOps-driven pipeline on Azure Kubernetes Service (AKS).

Key contributions include provisioning cloud infrastructure using Terraform, containerizing ASP.NET Core and Angular applications, and managing continuous delivery via Helm and Argo CD. The system's architecture also emphasizes security and decoupling by implementing OAuth 2.0 with PKCE, opaque tokens for authentication, and Azure Managed Identities for secure resource access. By detailing the CI/CD workflows, architectural decisions, and observability setups, this report provides a comprehensive overview of the project's current state and outlines necessary steps for future load testing and environment validation.

Contents

Abstract i

1 Project Overview 1

1.1 Context and Objectives . . . 1

1.2 Scope and Methodology . . . 2

2 Repository Structure 3

2.1 Application Layer (`app`) . . . 3

2.2 Infrastructure as Code Layer (`infrastructure`) . . . 4

2.3 GitOps and Operations Layer (`deploy`) . . . 4

3 Application Architecture 5

3.1 Microservices Architecture . . . 5

3.2 Authentication Flow . . . 6

3.3 Application User Interface . . . 7

4 Infrastructure Architecture and Infrastructure as Code 9

4.1 System Architecture Overview . . . 9

4.2 Infrastructure as Code with Terraform . . . 10

4.2.1 Terraform State and Infrastructure Change Control . . . 10

4.3 AKS and Workload Lifecycle . . . 10

4.4 Networking, NSG, and Traffic Flow . . . 11

4.5 Key Vault and Managed Identity . . . 11

4.6 Workload and Runtime Security . . . 12

4.7 Storage, Disk, Certificates, and Backup . . . 12

4.8 Kubernetes Deployment and Helm . . . 13

5 CI/CD Pipeline and GitOps 14

5.1 Application CI Pipeline . . . 14

5.2 Infrastructure Pipeline . . . 15

5.3 GitOps with Argo CD . . . 16

6 System Observability and Resilience **19**

6.1 Overall Monitoring Workflow 19

6.2 Measurement and Investigation 20

7 Quality Control and DevOps Assessment **24**

7.1 Pre-release Quality Gates 24

7.2 Workload and Monitoring Testing 24

7.3 Achievements 24

7.4 Limitations and Future Work 25

8 Conclusion **26**

List of Figures

3.1 Application architecture comprising two frontends, five backend services, MariaDB, and Valkey . . . 6

3.2 OAuth 2.0 Authorization Code flow with PKCE, opaque tokens, centralized verification, and refresh/revocation . . . 7

3.3 CustomerApp interface for end-users . . . 8

3.4 AdminPortal interface for administrators . . . 8

4.1 Architecture of Azure, AKS, GitOps, observability, and backup systems . . . 9

5.1 CI/CD Pipeline . . . 15

5.2 Argo CD Root App managing foundational applications and application services 17

6.1 Metrics and log data sources across the entire monitoring ecosystem . . . 19

6.2 Overall monitoring workflow from worker node to Grafana . . . 20

6.3 Prometheus query in Grafana tracking node memory . . . 21

6.4 NGINX Ingress dashboard with request rate and HTTP statuses . . . 21

6.5 Application log queries in Grafana Explore via Loki . . . 22

6.6 NGINX Ingress access log queries in Grafana Explore . . . 22

List of Tables

3.1 Primary Application Components 5

4.1 Workload identities and their usage purposes 12

5.1 Significance of control layers in the pipeline 18

Chapter 1.

Project Overview

1.1. Context and Objectives

When a system comprises multiple services, manual deployments often lead to environment inconsistencies, difficult configuration tracking, and potential secret exposure. This project addresses these challenges by transforming infrastructure, release processes, and operations into version-controlled code that can be reviewed and reliably repeated. MyTaskTracker serves as the workload illustrating these practices on the Azure and Kubernetes platforms.

On the application side, MyTaskTracker consists of two Angular 18 frontends (CustomerApp and AdminPortal) and five ASP.NET Core .NET 8 backends: AuthService, TaskService, ProjectService, CommentService, and NotificationService. Relational data is stored in MariaDB, while Valkey maintains the opaque token states for AuthService. This microservices architecture necessitates a unified DevOps pipeline to package, configure, deploy, and observe each component without tightly coupling them.

The technical objectives of the internship are to build a DevOps pipeline with the following characteristics:

- Standardize the packaging and deployment of two frontends, five backends, MariaDB, and Valkey on a unified Kubernetes platform.
- Declare Azure and AKS infrastructure using Terraform, establishing controllable state and plan/apply workflows.
- Package workloads into Docker images, ensure quality gates, and release immutable images tagged by Git SHA.
- Deploy applications using Helm and GitOps via Argo CD instead of applying manifests manually.
- Utilize OIDC, Managed Identities, Azure Key Vault, and NetworkPolicies to reduce static secrets and enforce the principle of least privilege.
- Monitor metrics, logs, and alerts, and execute backup and restore drills for post-deployment operations.

1.2. Scope and Methodology

The scope encompasses the application source code in the `app` directory, deployment manifests in `deploy`, Terraform configurations in `infrastructure`, and GitHub Actions workflows. The review prioritizes executable code over legacy documentation; therefore, the report distinguishes between existing source code functionality, expected deployment configurations, and outcomes that require direct verification.

Chapter 2.

Repository Structure

The repository is organized into three primary layers. The app directory contains the application source code; `infrastructure` houses Infrastructure as Code (IaC) for Azure; and `deploy` holds the desired state for the applications and platform tools running on Kubernetes. This separation ensures that interface or business logic changes do not intertwine with infrastructure and operational modifications.

The source code is managed at GitHub – kudoshinichi2k5/MyTaskTracker. This link serves as the reference point for the source, CI/CD workflows, Terraform, and GitOps manifests discussed in this report.

```
MyTaskTracker/
|- app/ # Application source and local development
| |- Backend/ # 5 ASP.NET Core .NET 8 services and tests
| |- Frontend/ # CustomerApp and AdminPortal (Angular 18)
| |- infra/mariadb/init/ # Initialize database and service users locally
| ‘- docker-compose.yml # Local stack, migrations, and seeding
|- infrastructure/ # Infrastructure as Code with Terraform
| |- environments/ # Dev persistent, dev workloads, staging, prod
| |- modules/ # ACR, AKS, networking, Key Vault, identity, storage
| ‘- scripts/ # Restore drills and operational scripts
|- deploy/ # Kubernetes and GitOps desired state
| |- charts/microservice/ # Generic Helm chart for all services
| |- values/base/ # Service-specific values
| |- environments/dev/ # Values for ingress, monitoring, Loki, Alloy
| |- argocd-apps/ # Argo CD Applications and ApplicationSets
| |- security/ # NetworkPolicies
| ‘- observability/ # Dashboards and alert rules
‘- .github/workflows/ # Application CI and Terraform CI
```

2.1. Application Layer (app)

The `app/Backend` directory contains the AuthService, TaskService, NotificationService, ProjectService, and CommentService. Each service features its own `Program.cs`, endpoints, DbContext, migrations, Dockerfile, and configurations; test projects are located in `app/Backend/Tests`. `app/Frontend` houses two independent Angular SPAs: CustomerApp for end-users and AdminPortal for administrators. The `docker-compose.yml` file orchestrates these services along-

side MariaDB, Valkey, migration jobs, and frontends to facilitate local development.

2.2. Infrastructure as Code Layer (`infrastructure`)

The `infrastructure/environments/dev/persistent` directory manages long-lifecycle resources such as ACR, DNS Zones, public IPs, storage accounts, and related Terraform states. The `infrastructure/environments/dev/workloads` directory consumes outputs from the persistent state to provision VNets, AKS, Key Vaults, workload identities, and role assignments. Reusable components are placed in `infrastructure/modules`, allowing staging or production environments to re-instantiate configurations without duplicating resource definitions.

2.3. GitOps and Operations Layer (`deploy`)

The `deploy/charts/microservice` directory contains the standardized Helm chart for backends and frontends. Service-specific values reside in `deploy/values/base`, while environment-specific values are kept in `deploy/environments`. Argo CD monitors the configurations defined in `deploy/argocd-apps`. NetworkPolicies, observability dashboards, and alert rules are managed alongside the application configuration, making the repository the single source of truth for the cluster's desired state.

Chapter 3.

Application Architecture

3.1. Microservices Architecture

MyTaskTracker is decoupled into small, functionally specific services. The user interface comprises two Angular 18 applications: *CustomerApp* for end-users and *AdminPortal* for administrators. These applications strictly handle the browser experience; business logic and data persistence are processed at the backend.

The backend consists of five ASP.NET Core .NET 8 Web APIs: *AuthService*, *TaskService*, *ProjectService*, *CommentService*, and *NotificationService*. Each service has its own Dockerfile, configurations, DbContext, and database migrations. MariaDB serves as the relational data layer, with each service owning its specific database to prevent shared tables and reduce direct coupling between business domains. Valkey does not store business data; instead, it maintains session states and tokens for the AuthService.

Service decoupling does not imply disjointed operations. Frontends invoke APIs based on functionality; resource services utilize the AuthService to verify identities rather than managing login sessions independently. Upon deployment to Kubernetes, each frontend and backend is packaged into a separate container image, allowing them to be released, scaled, and monitored independently.

Table 3.1: Primary Application Components

Component	**Role**
CustomerApp & AdminPortal	Two Angular 18 Single Page Applications serving end-users and administrators.
AuthService	Handles registration, login, token issuance, and revocation; provides a token verification endpoint.
TaskService	Manages tasks and aggregates task data for the admin interface.
ProjectService	Manages projects and task-to-project associations.
CommentService & NotificationService	Manages task-based comments and user notifications.
MariaDB	Relational database storage, logically segregated per service.
Valkey	Stores opaque token states and refresh sessions for the AuthService.

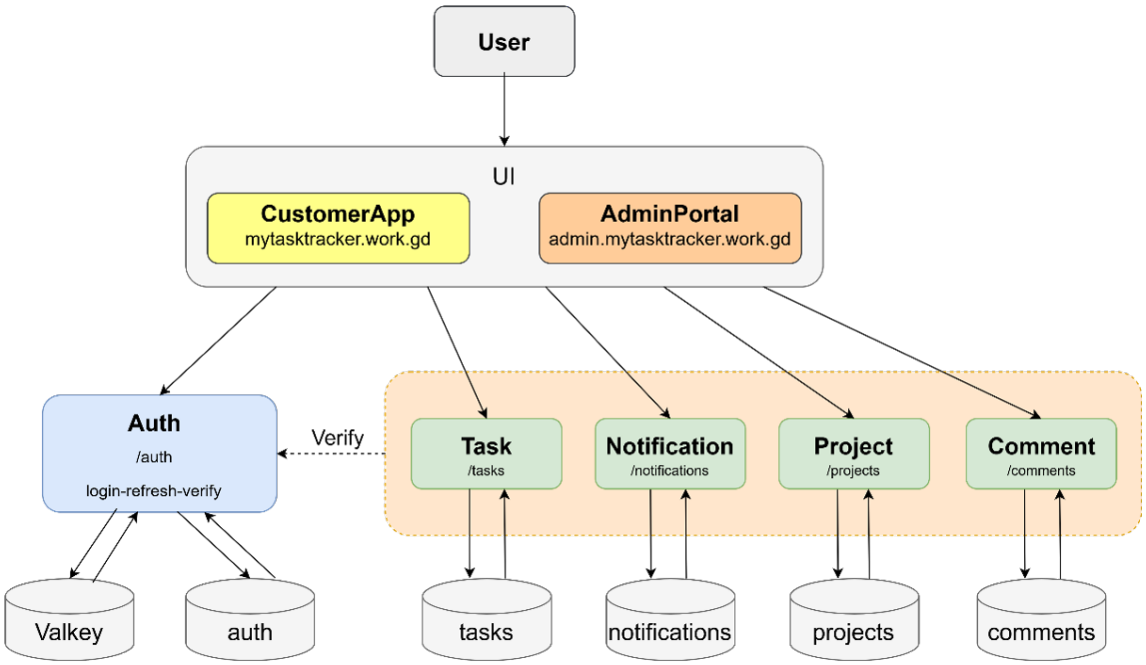


Figure 3.1: Application architecture comprising two frontends, five backend services, MariaDB, and Valkey

3.2. Authentication Flow

Figure 3.2 illustrates the OAuth 2.0 Authorization Code with PKCE mechanism for the two SPAs. The AuthService acts as the Authorization Server. Valkey exclusively stores server-side states and token hashes, while the access token dispatched to the client is an opaque token, meaning it contains no self-decodable claims.

A. Login and Token Issuance. The CustomerApp or AdminPortal generates a `state`, `code_verifier`, and `code_challenge` (S256), then redirects the browser to `/connect/authorize`. The AuthService stores the authorization request in Valkey and displays the login page. Upon successful credential validation, the service saves a single-use authorization code in Valkey and redirects back to the SPA's callback with the `code` and `state`. The SPA sends the `code` and `code_verifier` to `POST /connect/token`. The AuthService verifies the PKCE, consumes the single-use code, generates a random access token and refresh token, stores their hashes in Valkey, and returns the token pair to the SPA.

B. API Calls and Token Verification. The SPA attaches the opaque access token to the `Authorization: Bearer <token>` header when invoking the TaskService, ProjectService, CommentService, or NotificationService. The resource service does not decode the token itself, nor does it access Valkey directly; instead, it calls `GET /verify` on the AuthService. The AuthService hashes the received token, queries Valkey, and returns the status (`active`), user,

roles, and remaining validity. The resource service processes the request and returns API data only if the token is valid.

C. Expiration, Refresh, and Logout. When a resource service returns `401`, the SPA sends the refresh token to `POST /connect/token` to obtain a new token pair, then retries the failed request. The refresh token is rotated, and the old session is consumed to mitigate replay attacks. If the refresh fails, the SPA clears the local session and redirects the user to the login flow. During logout, the SPA invokes `/connect/revocation` to allow the AuthService to revoke the access token or refresh session in Valkey.

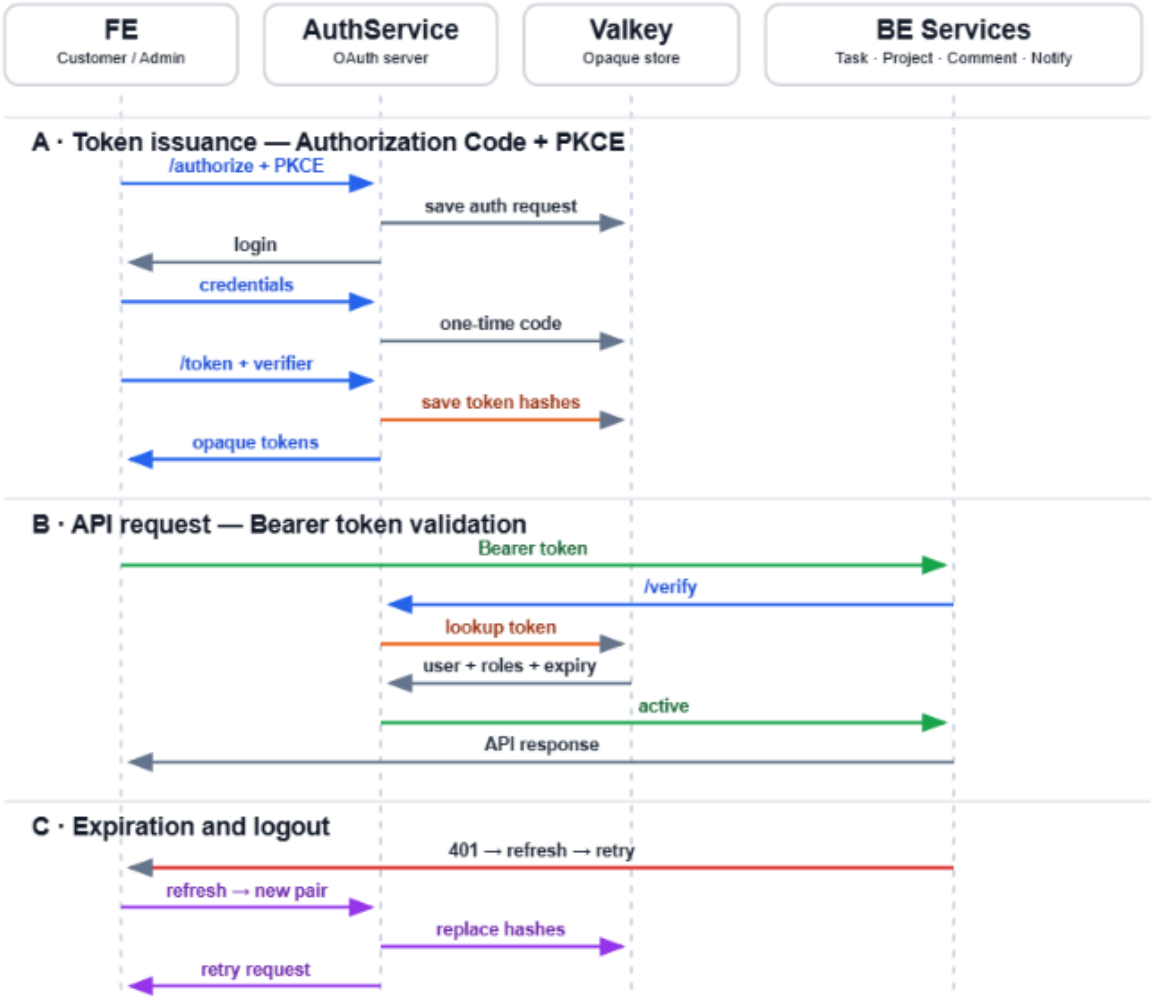


Figure 3.2: OAuth 2.0 Authorization Code flow with PKCE, opaque tokens, centralized verification, and refresh/revocation

3.3. Application User Interface

CustomerApp is designed for daily user operations, whereas AdminPortal facilitates administration and monitoring. The following images are provided to illustrate the interface without

delving deeply into the business logic of each screen.

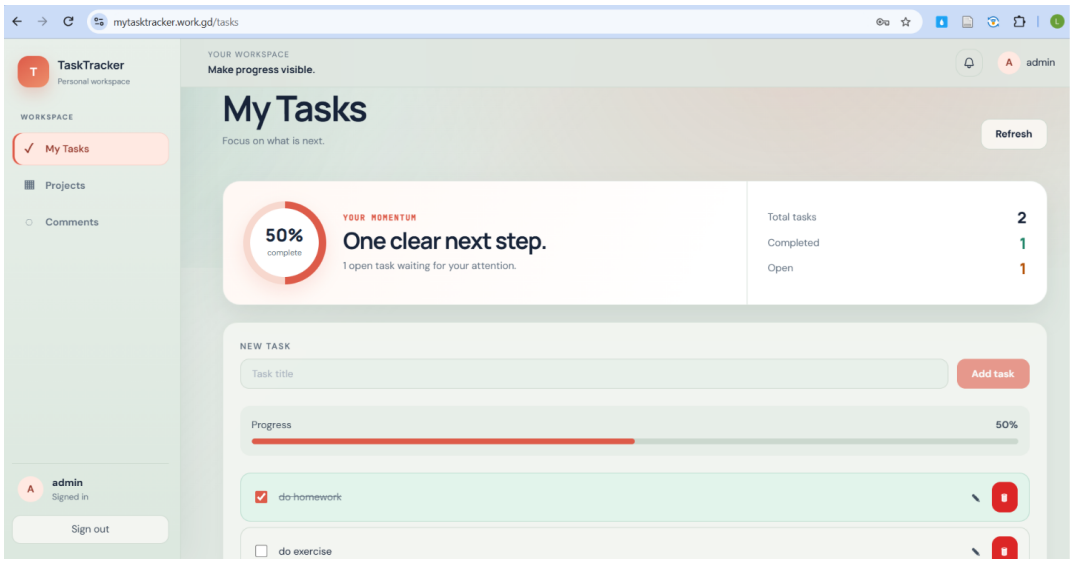


Figure 3.3: CustomerApp interface for end-users

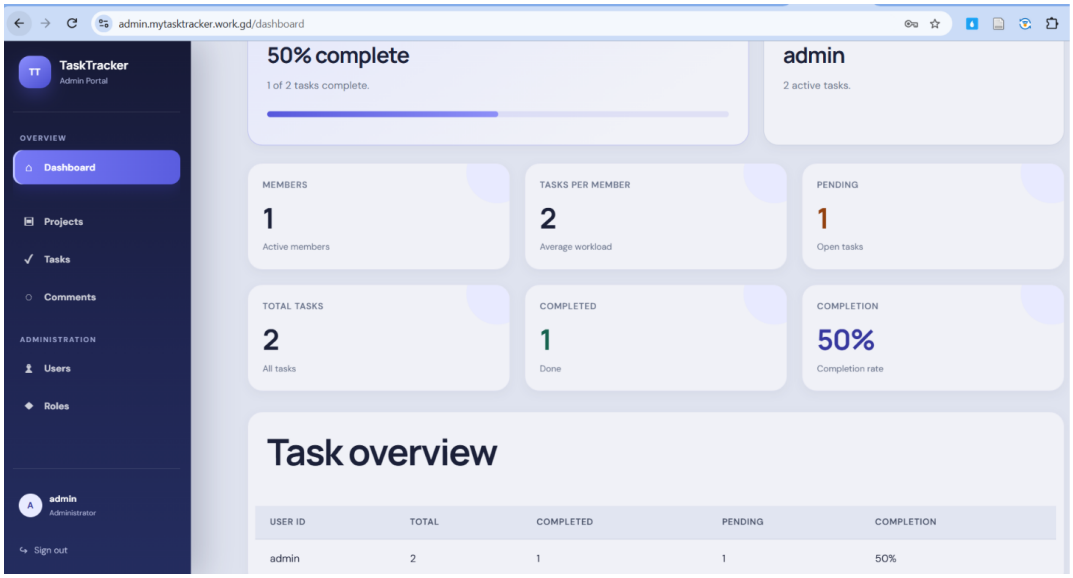


Figure 3.4: AdminPortal interface for administrators

Chapter 4.

Infrastructure Architecture and Infrastructure as Code

4.1. System Architecture Overview

The architecture revolves around Azure and AKS. Terraform manages cloud resources; GitHub Actions builds artifacts and images; Argo CD synchronizes configurations from Git to Kubernetes. Applications run in the `dev` namespace, while monitoring, logging, and operational components are isolated in their respective namespaces.

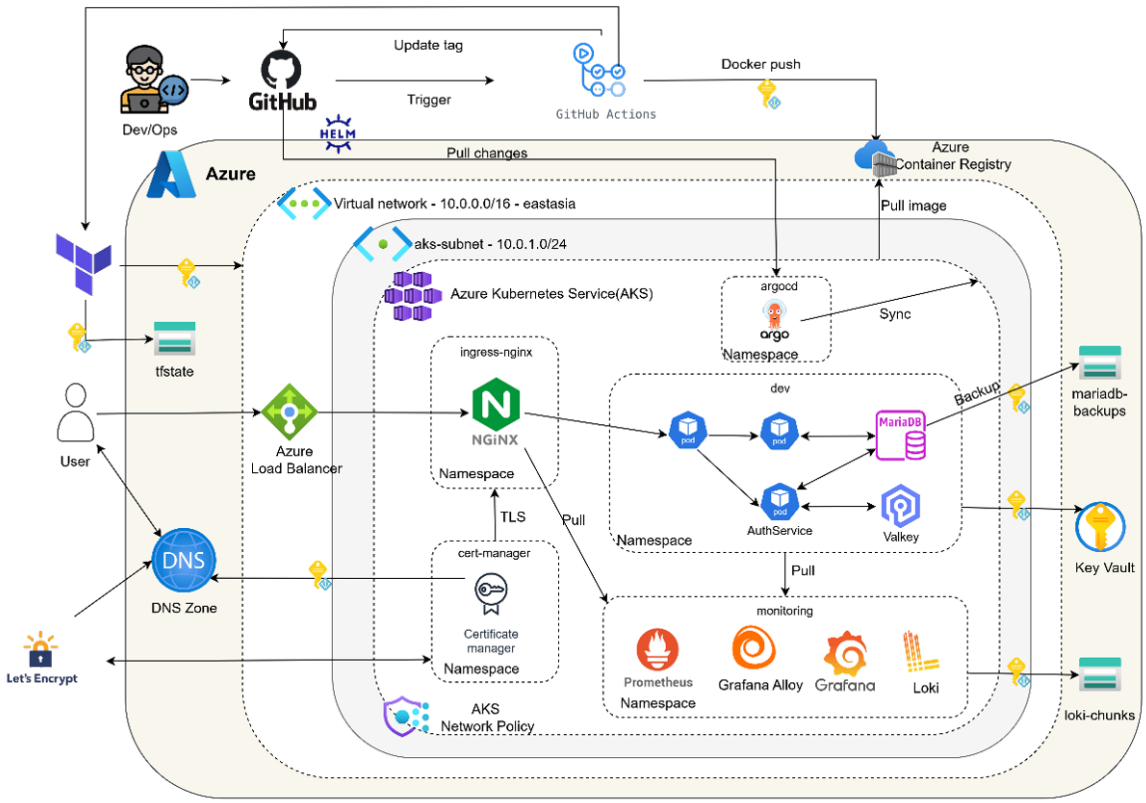


Figure 4.1: Architecture of Azure, AKS, GitOps, observability, and backup systems

Figure 4.1 illustrates the overall operational flow. Terraform provisions or updates the VNet, AKS, ACR, Key Vault, DNS Zone, and storage accounts. Upon source code changes, GitHub Actions builds the image and pushes it to ACR; Argo CD inside AKS pulls Git changes and syncs the Helm release. At runtime, users route through DNS, Azure Load Balancer, and NGINX Ingress to the `dev` namespace. Prometheus, Grafana, Loki, and Alloy operate within the `monitoring` namespace; MariaDB backups and Loki chunks are shipped to dedicated Blob Storage accounts.

4.2. Infrastructure as Code with Terraform

The entire Azure infrastructure is described using Terraform. The source is split into two states to clarify resource lifecycles. The persistent state contains stable components such as resource groups, Azure Container Registry, DNS Zones, public IPs, backup storage accounts, Loki storage accounts, and their corresponding Terraform states. The workload state reads persistent outputs to provision the VNet, AKS, Key Vault, managed identities, Azure RBAC assignments, and cluster-dependent Kubernetes components.

Splitting the state prevents accidental deletion of data-bearing resources when workload changes are required. It also enforces deployment sequence: foundational resources must exist before AKS and workloads can reference them.

4.2.1. Terraform State and Infrastructure Change Control

Terraform states are stored in Azure Storage, and the workload state retrieves persistent outputs via `terraform_remote_state`. This forms a crucial link between the two infrastructure layers: workloads do not hard-code names for ACR, DNS Zones, or Storage Accounts; they consume managed outputs. Infrastructure CI utilizes OIDC, eliminating the need for static Azure credentials, and separates the plan phase on pull requests from the apply phase on the main branch. When both persistent and workloads change in a single pull request, the workloads plan serves merely as a reference; after the persistent state is applied, the pipeline generates a new plan from main to prevent applying stale states.

4.3. AKS and Workload Lifecycle

AKS utilizes a System Assigned Identity, Azure CNI, network policies, a Standard Load Balancer, an OIDC issuer, and Workload Identity. The Key Vault CSI driver is enabled to allow workloads to mount or sync secrets based on their identity. Node pools autoscale between min/max thresholds defined in Terraform; at the application layer, HPA scales pod replicas based on CPU metrics given defined resource requests. These two autoscaling layers solve distinct problems: HPA adjusts pod counts, while the cluster autoscaler adjusts node capacity when the scheduler lacks space.

When a new release syncs, Deployments execute rolling updates natively in Kubernetes. Readiness probes keep new pods out of the Service endpoints until they are fully initialized; liveness probes assist the kubelet in restarting unresponsive pods. Migrations are decoupled into Jobs rather than allowing replicas to self-migrate, mitigating the risk of concurrent schema changes.

4.4. Networking, NSG, and Traffic Flow

The `dev` environment provisions a VNet (`10.0.0.0/16`), consisting of an `aks-subnet` (`10.0.1.0/24`) for AKS and a `db-subnet` (`10.0.2.0/24`) reserved for the data tier. Users access the system via DNS, Standard Static Public IPs, Azure Load Balancer, and NGINX Ingress before requests reach ClusterIP Services and application pods. Consequently, backend services are never directly exposed to the Internet.

A Network Security Group (NSG) is attached to the `db-subnet`. Existing rules allow TCP 3306 from the AKS subnet, followed by a rule blocking remaining inbound traffic within the VNet. At the Kubernetes layer, the `dev` namespace enforces a default-deny ingress policy; specific NetworkPolicies selectively open essential flows, such as ingress-nginx to applications, backends to AuthService, and workloads to MariaDB or Valkey. Egress is not currently default-denied, ensuring pods can resolve DNS and invoke Azure services.

4.5. Key Vault and Managed Identity

Key Vault securely stores service-specific connection strings, MariaDB passwords, and Valkey passwords. Passwords are generated by Terraform, removing the need to write secrets directly into the repository. Helm charts generate a `SecretProviderClass`; the Azure Key Vault CSI driver fetches secrets based on the workload identity and syncs them into Kubernetes Secrets to be injected into containers as environment variables.

Managed Identities replace static access keys or client secrets. Permissions are strictly scoped: backends read Key Vault using *Key Vault Secrets User*; cert-manager utilizes *DNS Zone Contributor*; backup jobs and Loki possess *Storage Blob Data Contributor*. GitHub Actions also utilizes federated OIDC identities for Azure login, ensuring workflows never store long-lived Azure passwords.

Table 4.1: Workload identities and their usage purposes

Entity	Accessed Resource	Purpose
Backend services	Azure Key Vault	Read connection strings and service-specific secrets.
cert-manager	Azure DNS Zone	Create or verify DNS records for TLS certificate issuance.
db-backup CronJob	Azure Blob Storage	Push MariaDB dumps and checksum files to a private container.
Loki	Azure Blob Storage	Store chunks and index data according to object storage configurations.
GitHub Actions	Azure, ACR, Terraform state	Build/push images and execute infrastructure pipelines via OIDC.

4.6. Workload and Runtime Security

Secrets traverse through Key Vault, Workload Identity, and SecretProviderClass rather than being hard-coded in manifests. The `dev` namespace enforces default-deny ingress and strictly opens defined communication paths. NGINX Ingress applies rate limiting for AuthService; services operate with explicitly defined liveness/readiness probes and resource requests/limits. Because health endpoints currently primarily reflect the API process status, implementing dependency health checks for MariaDB and Valkey is a necessary improvement so readiness accurately reflects serving capability.

4.7. Storage, Disk, Certificates, and Backup

Prometheus is configured with a 2Gi persistent volume for short-term metrics storage; Loki SingleBinary uses a 5Gi persistent volume as a local workspace. MariaDB and Valkey utilize storage provisioned by Helm charts and the cluster provisioner. For long-term retention, Blob Storage is segregated for MariaDB backups and Loki chunks. Backup storage is subject to lifecycle policies: moving to the Cool tier after 7 days and deletion after 30 days, as defined by Terraform.

Cert-manager employs workload identity to manipulate the Azure DNS Zone during the DNS challenge process. Certificates are subsequently stored in TLS secrets, which NGINX Ingress utilizes for public hosts. The backup CronJob generates dumps, manifests checksums,

and uploads them via the Azure REST API; restore drills download the latest backup, verify the SHA-256 hash, and restore to a temporary database for validation.

4.8. Kubernetes Deployment and Helm

A generic Helm chart produces Deployments, Services, Ingresses, HPAs, migration Jobs, ServiceMonitors, SecretProviderClasses, and backup CronJobs based on service-specific values. Migration Jobs run with the `-migrate` parameter before service initialization. HPA functions reliably only when containers define resource requests; thus, services enabling HPA must declare appropriate CPU/memory requests. Argo CD's sync waves also require strict control, as prematurely created HPAs or dependent resources may throw "Deployment not found" errors.

Chapter 5.

CI/CD Pipeline and GitOps

5.1. Application CI Pipeline

The `ci.yml` workflow utilizes `dorny/paths-filter` to selectively build modified components; the `build_all` option can forcibly trigger a full build. The pipeline comprises the following primary steps:

1. Scan secrets using Gitleaks.
2. Restore, build, and test .NET 8 across a backend matrix, collecting TRX/Cobertura reports.
3. Execute `npm ci`, build the dev environment, and run Jest for each Angular app.
4. Lint Helm, render Valkey, and dry-run charts with AuthService/CustomerApp values.
5. On the `main` branch, publish artifacts, build `linux/arm64` images, authenticate to Azure via OIDC, and push to ACR.
6. Update the image tag SHA in `deploy/values/base` and commit GitOps changes with `[skip ci]`.

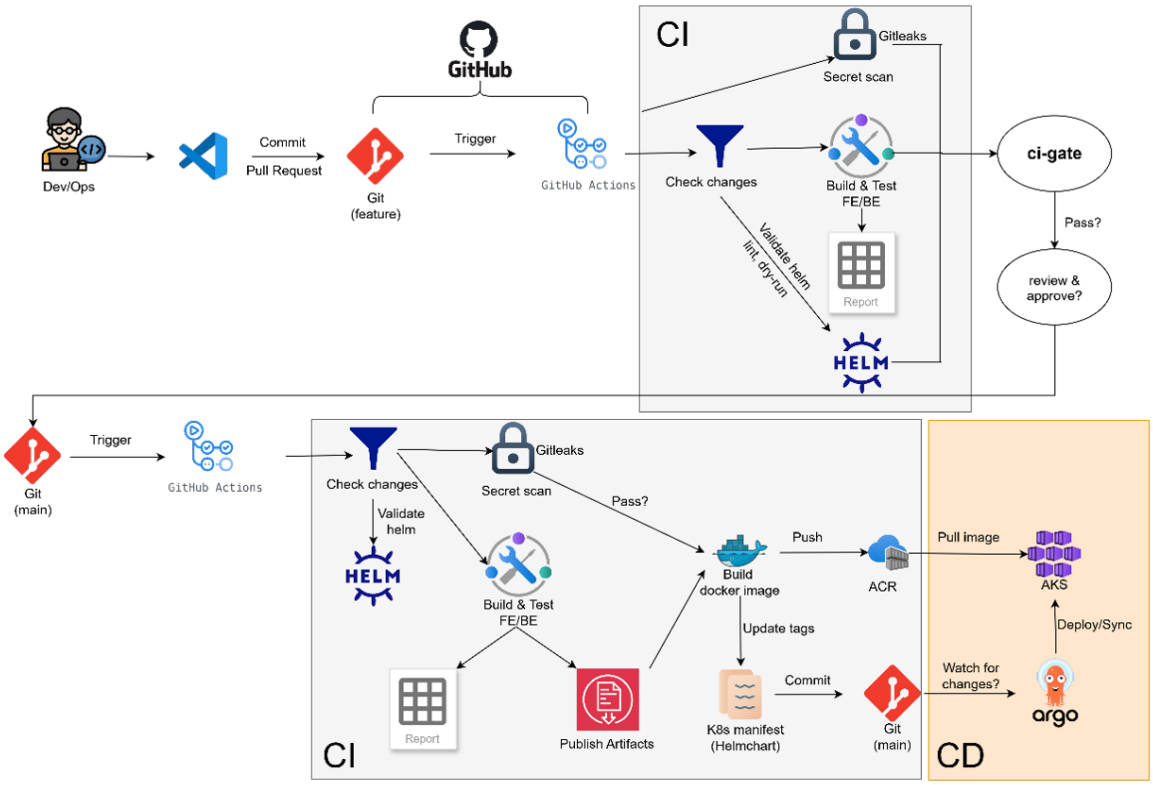


Figure 5.1: CI/CD Pipeline

As shown in Figure 5.1, pull requests navigate through validation steps—secret scanning, build/test, and Helm validation—to facilitate code review. Only when changes are merged into the main branch does the pipeline publish artifacts, build Docker images, push to ACR, and update image tags in Git. Argo CD detects these GitOps commits, pulls the corresponding images from ACR, and syncs the workloads to AKS. This decoupling ensures that unapproved pull requests cannot mutate the running release.

5.2. Infrastructure Pipeline

The infrastructure workflow processes Terraform exclusively when changes occur in persistent or workloads states. During a pull request, the pipeline executes Checkov, `terraform init`, `validate`, and `plan`. On the main branch, persistent states are applied first. In cases of concurrent changes across both states, workload plans are regenerated after persistent states are successfully written; only fresh plans derived from the main branch are eligible for application. This practice prevents the application of outdated plans that no longer align with remote-state outputs.

The pipeline also features an explicit, intentional destroy branch. Prior to destruction, the workflow fetches AKS credentials, instantiates a Job from the backup CronJob, and awaits its completion; if the job fails or times out, the destroy process halts. This acts as an in-source

safeguard, though its practical efficacy requires validation in the deployed environment.

5.3. GitOps with Argo CD

Argo CD fetches manifests using a pull-based model. A root application operates under the app-of-apps pattern; ApplicationSets deploy services based on discrete values; generic charts enforce resource standardization. Replacing image tags in Git following a successful build establishes an auditable history of changes while separating the permission to build images from the permission to apply resources directly to the cluster.

This model draws a definitive operational boundary: GitHub Actions is responsible for generating artifacts and updating desired states; Argo CD acts as the in-cluster agent responsible for pulling configurations and reconciling toward that state. Automated sync, self-healing, and pruning are configured for foundational applications like monitoring, Loki, and Alloy. If manual configuration drift occurs within the cluster, Argo CD detects it and reverts resources to the Git-defined configuration. However, pruning must be utilized cautiously, as removing resources from Git triggers their deletion from the cluster.

Helm values segregate global configurations, environment configurations, and service-specific configurations. A single chart can render Deployments for backends, NGINX runtimes for frontends, migration Jobs, or backup CronJobs depending on input values. This approach minimizes manifest duplication while allowing each service to define its specific image, ingress, secrets, probes, resources, and autoscaling rules.

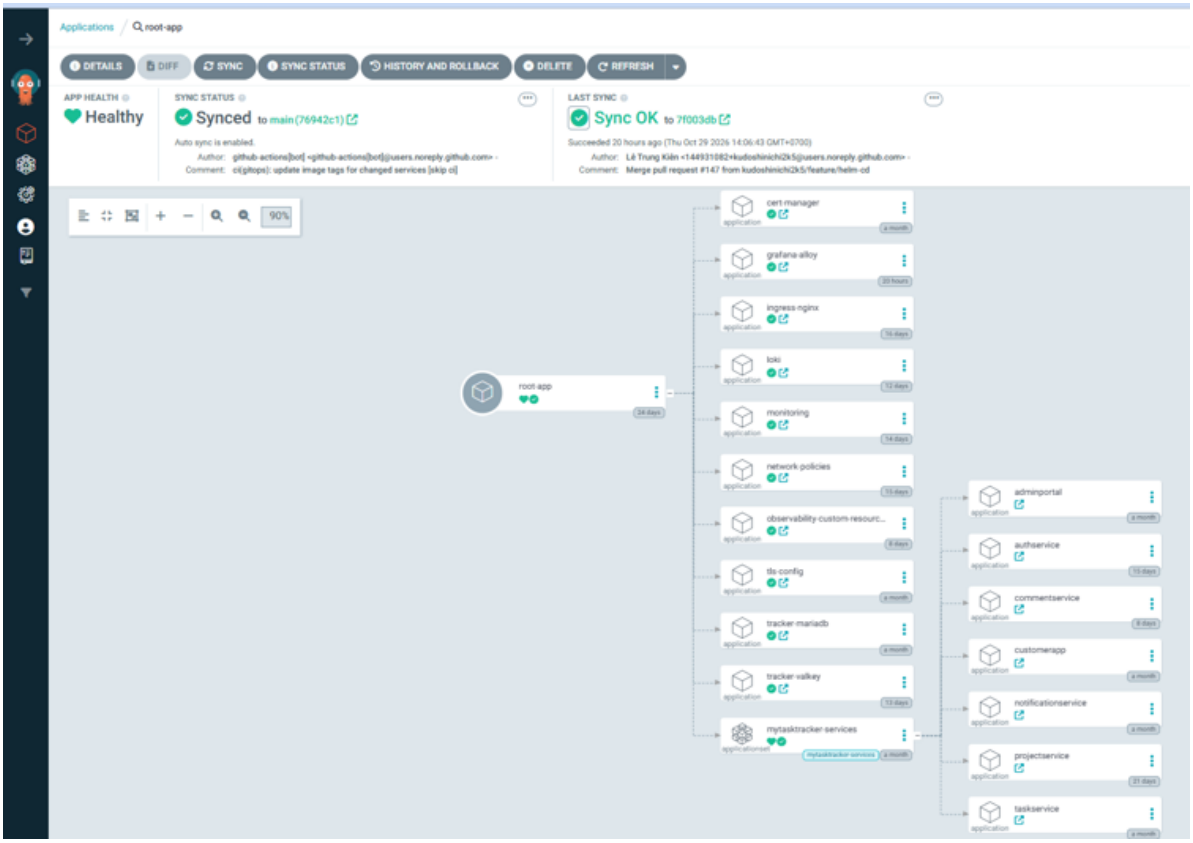


Figure 5.2: Argo CD Root App managing foundational applications and application services

The Root App image demonstrates the active app-of-apps model. The Root App oversees foundational applications such as cert-manager, ingress-nginx, Loki, Grafana Alloy, monitoring, NetworkPolicies, TLS configurations, MariaDB, and Valkey; concurrently, it manages ApplicationSets that spawn application services. Statuses of *Healthy*, *Synced*, and *Sync OK* serve as point-in-time evidence that the desired state in Git has been successfully reconciled into the cluster by Argo CD.

Table 5.1: Significance of control layers in the pipeline

Layer	**Control**
Source change	`paths-filter` reduces redundant builds; `build_all` accommodates mass releases.
Security	Gitleaks detects exposed secrets in code by scanning the commit history.
Quality	Backend tests, frontend Jest, and artifact test/coverage provide CI evidence.
Deployability	Helm lint/template catches chart or value errors prior to GitOps sync.
Release	OIDC prevents storing static Azure credentials in workflows; ACR tags images by Git SHA.
Operations	Argo CD synchronizes desired states from the repository to Kubernetes.

Chapter 6.

System Observability and Resilience

6.1. Overall Monitoring Workflow

System monitoring encompasses both metrics and logs, extending beyond Prometheus alone. Metric sources include backend services, NGINX Ingress, node-exporter, kubelet/cAdvisor, and kube-state-metrics. Log sources include `stdout/stderr` from backend containers and access logs from NGINX Ingress. The table below outlines the extraction components, the provided information, and the operational value of each source.

Source	Collection method and key information	Purpose in the report
Backend services	Prometheus scrapes /metrics for request count, status codes, latency, endpoints, and methods.	Application health and performance.
NGINX Ingress	Prometheus scrapes NGINX metrics for traffic, 2xx/4xx/5xx responses, latency, and active connections.	Gateway health and external traffic.
Node Exporter	Prometheus scrapes node metrics for worker-node CPU, memory, disk, and network usage.	Node infrastructure resources.
Kubelet / cAdvisor	Prometheus scrapes container metrics for Pod/container CPU, memory, network, and filesystem usage.	Container and Pod resource consumption.
kube-state-metrics	Prometheus scrapes Kubernetes object metrics, including Deployment, Pod, ReplicaSet, and HPA desired/current states.	Kubernetes workload status.
Backend container logs	Alloy collects stdout / stderr logs containing exceptions, requests, events, and error messages.	Application troubleshooting.
NGINX Ingress logs	Alloy collects NGINX access/error logs, including upstream errors and 404/502/504 responses.	Gateway and routing troubleshooting.
Loki	Alloy pushes logs to Loki, which stores and indexes them by namespace, Pod, container, and labels.	Centralized log search and correlation.
Azure Blob Storage	Loki stores log chunks and indexes in Azure Blob Storage.	Durable, long-term log retention.

Figure 6.1: Metrics and log data sources across the entire monitoring ecosystem

These sources are highly complementary. Metrics convey status, resource utilization, and trends over time; logs retain granular details of individual requests, exceptions, or access events. Thus, monitoring can traverse from high-level symptoms to specific root causes without relying solely on a single data type.

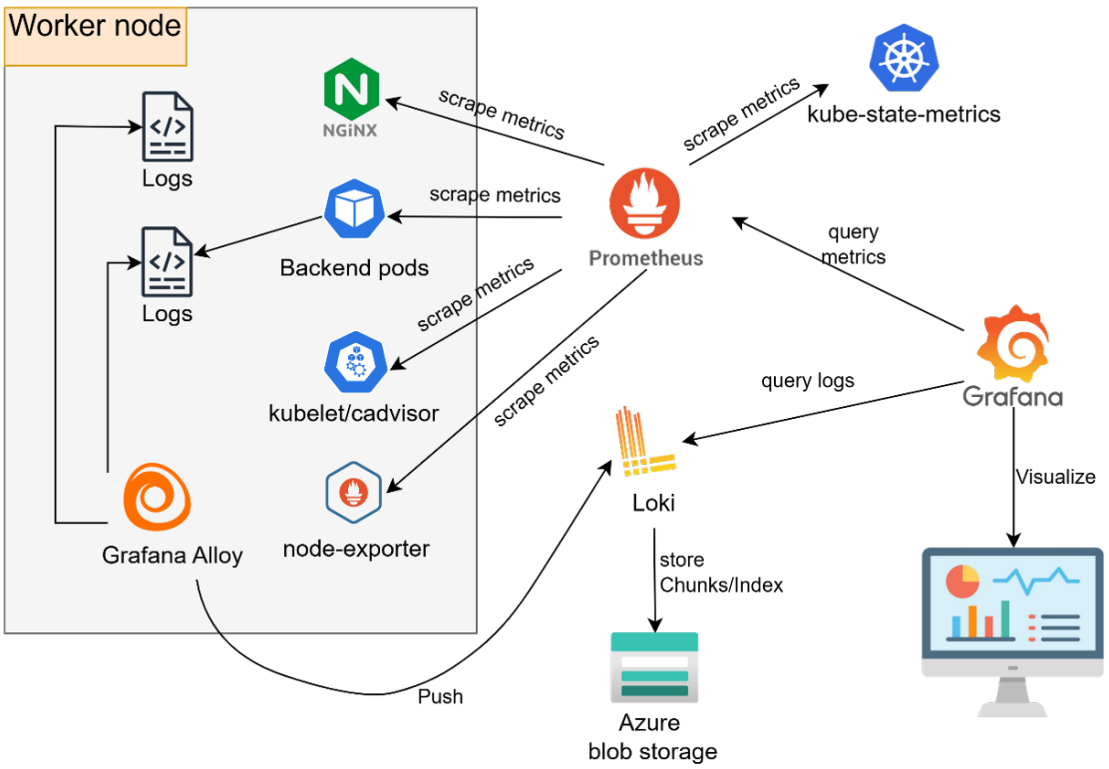


Figure 6.2: Overall monitoring workflow from worker node to Grafana

Figure 6.2 illustrates two parallel streams. Prometheus scrapes metrics from applications, NGINX, nodes, and Kubernetes components; Grafana queries Prometheus to render charts and dashboards. Simultaneously, Grafana Alloy runs as a DaemonSet on worker nodes, tailing Kubernetes logs and pushing them via the Loki Gateway. Loki compresses logs into chunks, indexes them by label and timestamp, and stores the data long-term in Azure Blob Storage. Grafana uses Loki as a datasource to query logs via LogQL. Consequently, metrics identify where an issue is occurring, while logs facilitate finding the detailed cause.

6.2. Measurement and Investigation

Grafana acts as the unified observation point for both metrics and logs. The Prometheus query image demonstrates tracking worker node memory over time. The NGINX dashboard displays request rates, HTTP statuses, ingresses, and HTTP methods, ideal for early detection of 4xx/5xx errors or unusual traffic anomalies.

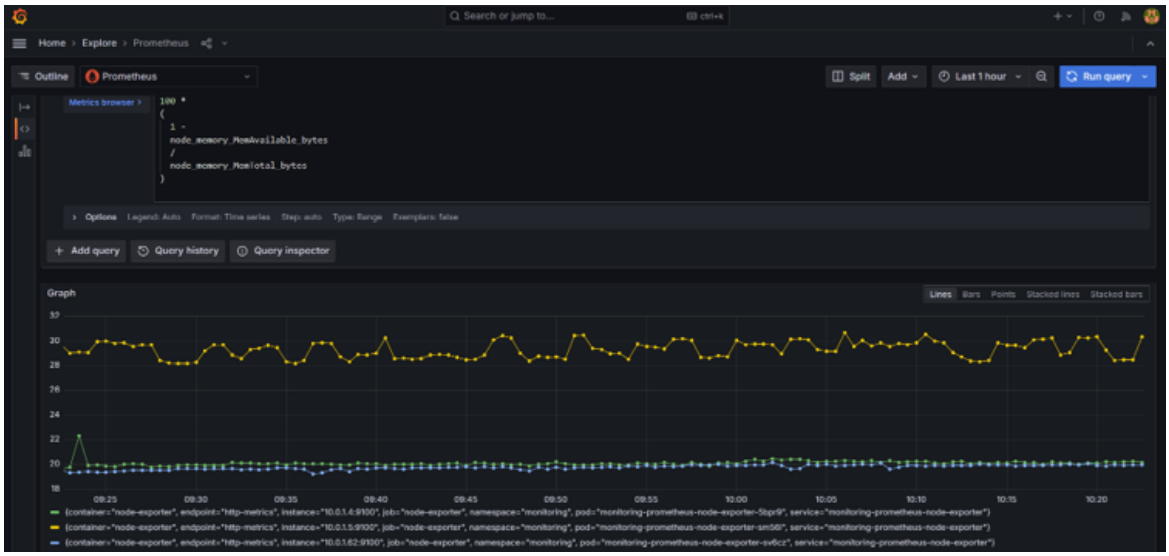


Figure 6.3: Prometheus query in Grafana tracking node memory

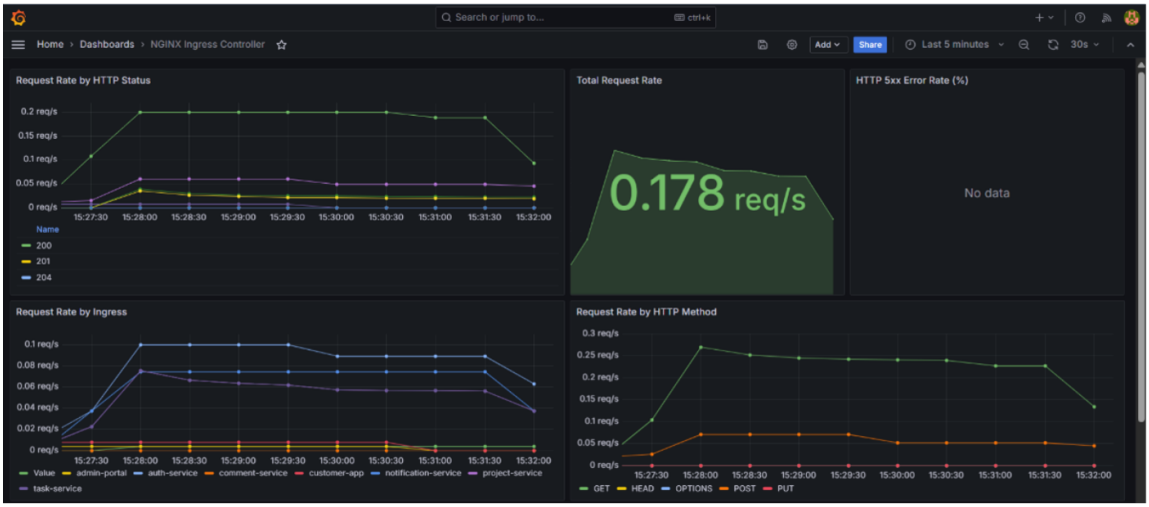


Figure 6.4: NGINX Ingress dashboard with request rate and HTTP statuses

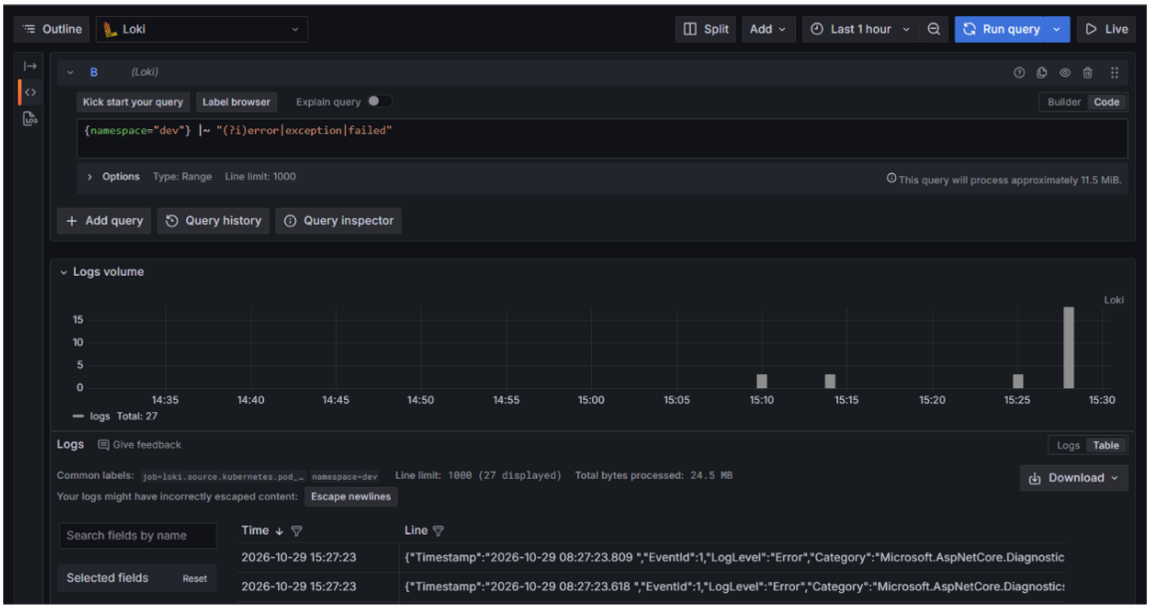


Figure 6.5: Application log queries in Grafana Explore via Loki

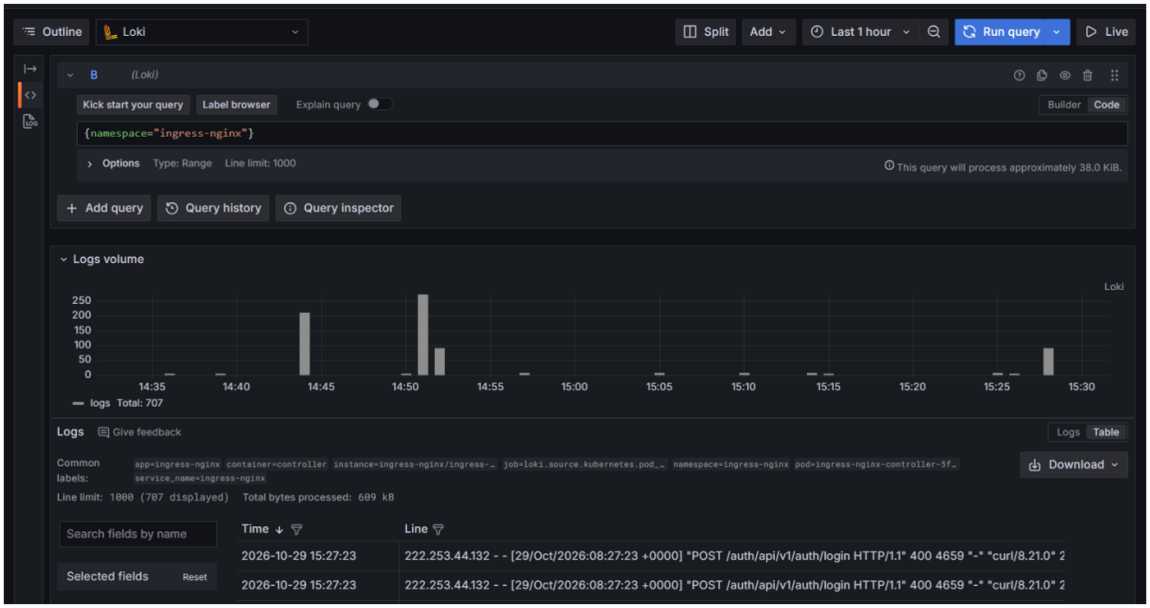


Figure 6.6: NGINX Ingress access log queries in Grafana Explore

The two Grafana Explore images supply investigatory data. Application logs in the `dev` namespace assist in locating exceptions, request traces, or business logic errors. Access logs in the `ingress-nginx` namespace aid in inspecting incoming gateway requests, status codes, upstreams, and routing errors. Incident response typically begins with request/latency or resource charts, followed by narrowing the timeframe and namespace to locate relevant logs.

Monitoring is only meaningful when verified. The recommended scenario involves generating controlled successful and failed requests, then cross-referencing request rates, status codes, and latencies in Grafana; subsequently, querying logs at the same timestamp in Loki confirms

labels and log content. With HPA, simulated loads can be generated in a staging environment to observe CPU requests, replica counts, and HPA events. Intentional loads or errors should not be executed directly on production environments.

The repository includes a PrometheusRule alert, `IngressHigh5xxRate`, which triggers a warning when the 5xx rate of NGINX Ingress exceeds 5% consistently for one minute. The Alertmanager is currently disabled in dev values, meaning the rule can be evaluated, but notification channels require further configuration for proactive alerting. Regarding data, restore drills must be run periodically: verifying that CronJobs create backups, checking blobs and checksums, restoring to a temporary database, and recording recovery times and the latest backup timestamp. These metrics constitute the foundation for declaring RTO/RPO, moving beyond relying solely on manifests.

Chapter 7.

Quality Control and DevOps Assessment

7.1. Pre-release Quality Gates

The application pipeline does not push images to the registry immediately upon source changes. Gitleaks scans for exposed secret risks; backends are restored, built, tested, and report TRX/Cobertura results; frontends run `npm ci`, build, and Jest. If deployment configurations change, the pipeline executes Helm lint and Helm template with actual values. These steps isolate source errors, dependency issues, and manifest defects before they reach the cluster.

Terraform CI integrates Checkov, `terraform validate`, and `terraform plan`. Plans on pull requests enable infrastructure change reviews prior to merging. For a multi-state system, ignoring outdated plans when remote-state outputs change is a more critical control than simply running automated `terraform apply`.

7.2. Workload and Monitoring Testing

Five backend test projects validate partial application quality prior to release. AuthService and TaskService feature integration tests via `WebApplicationFactory<Program>`; Testing suites utilize EF Core InMemory and mock token stores, eliminating dependencies on Docker, MariaDB, or Valkey. These fast CI tests do not replace authentic dependency checks in staging.

Post-deployment, testing must shift to an operational perspective: verifying health/readiness, Deployment/Pod/HPA statuses, Prometheus scrape targets, dashboard metrics, log labels in Loki, and 5xx alerts at the ingress. When databases or charts are modified, migration Jobs and Argo CD sync statuses are critical acceptance check criteria.

7.3. Achievements

- Standardized Azure and AKS infrastructure via Terraform, segregating persistent states from workload states.
- Constructed a release pipeline from source through CI, ACR, GitOps values, and Argo CD reconciliation.
- Implemented OIDC and Managed Identities to eliminate static secrets in GitHub Actions and Kubernetes workloads.

- Standardized manifests using Helm charts, environment-specific values, migration Jobs, probes, HPAs, and NetworkPolicies.
- Deployed an observability stack (Prometheus, Grafana, Loki, Alloy), dashboards-as-code, and ingress alert rules.
- Prepared MariaDB backups, Blob Storage lifecycles, and restore drills with checksum verification.

7.4. Limitations and Future Work

The following items should be prioritized in subsequent phases:

1. Resolve TestServer hostname issues to introduce integration testing endpoints for ProjectService and CommentService.
2. Integrate dependency health checks for MariaDB and Valkey to ensure readiness accurately reflects serving capability.
3. Execute end-to-end, load, and security testing in a staging environment; document results rather than inferring from configurations.
4. Review data synchronization between ProjectService and TaskService during attach/detach operations, handling unilateral failures and distributed consistency.
5. Validate HPA, sync waves, backups, and restore drills on an actual cluster; record quantifiable SLOs, RTOs, and RPOs.
6. Expand observability with distributed tracing and business dashboards tracking task completion rates.

Chapter 8.

Conclusion

This project concentrates on building DevOps capabilities for MyTaskTracker: transforming infrastructure, deployment, secrets, policies, monitoring, and backups into reviewable, repeatable, and traceable configurations within a repository. The microservices application serves as the illustrative workload; the true value of the system lies in the strictly controlled path from source code to the desired state on AKS.

Through this project, knowledge is solidified across Docker, Terraform, Azure, AKS, Kubernetes, Helm, Argo CD, GitHub Actions, OIDC, Key Vault, Prometheus, Grafana, Loki, and backup/restore strategies. The subsequent phase must prioritize direct verification in staging, perfecting dependency health checks, measuring SLO/RTO/RPO, and periodically executing load tests, monitoring checks, backups, and restore drills.